

Event-Driven Architecture

System Design Deep Dive - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	The Traditional Synchronous Model and its Problems	Why do we need something else?
2	What is Event-Driven Architecture? What is an Event?	How do services talk without calling each other?
3	The Four Components: Producer, Broker, Consumer, Topic	Who does what?
4	Event Processing Patterns	How much data should an event carry?
5	Failure Handling and Delivery Semantics	What happens when something breaks?
6	EDA in Real Systems	Who uses it and when should I?
7	Cheat Sheet and Interview Q&A	What must I remember?

1. The Traditional Synchronous Model

To understand event-driven architecture (EDA), first look at the **traditional synchronous architecture**, the problems it has, and how EDA solves them.

1.1 How an order is processed synchronously

A client places an order. The **order service** depends on several other services: **payment**, **inventory** and **notification**. It calls them one after another and **waits for each to finish** before moving to the next.

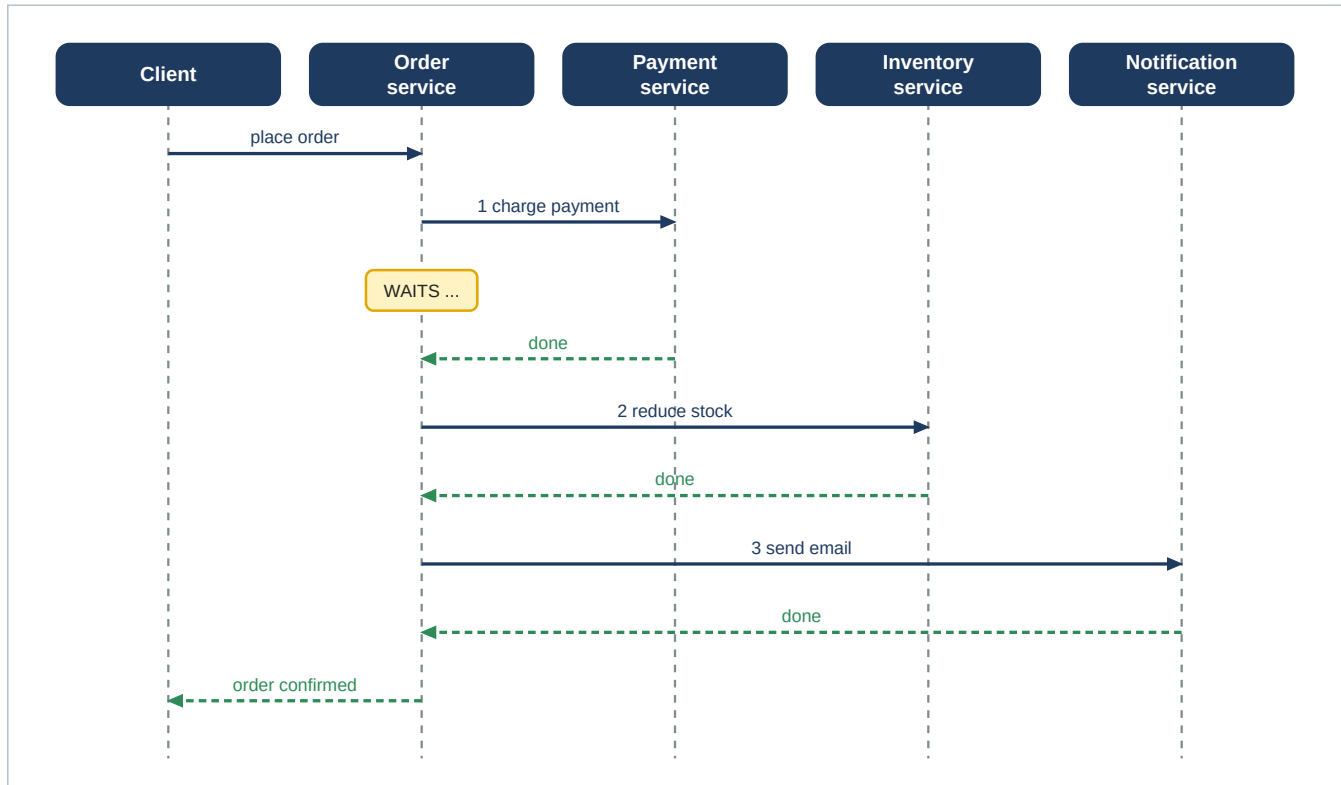


Figure 1: Synchronous order processing: a chain of blocking calls

- The client places an order; the **order service** is called.
- It calls the **payment service** and **waits** for it to complete.
- Then it calls the **inventory service**, which reduces stock, and waits again.
- Then it calls the **notification service**, which sends an email or message, and the order is complete.

EXAMPLE

You buy headphones on a shopping site. Behind the scenes: charge your card (wait), reduce stock from 10 to 9 (wait), send 'Order confirmed' email (wait), and only then does your screen say 'Done'.

1.2 Three major problems

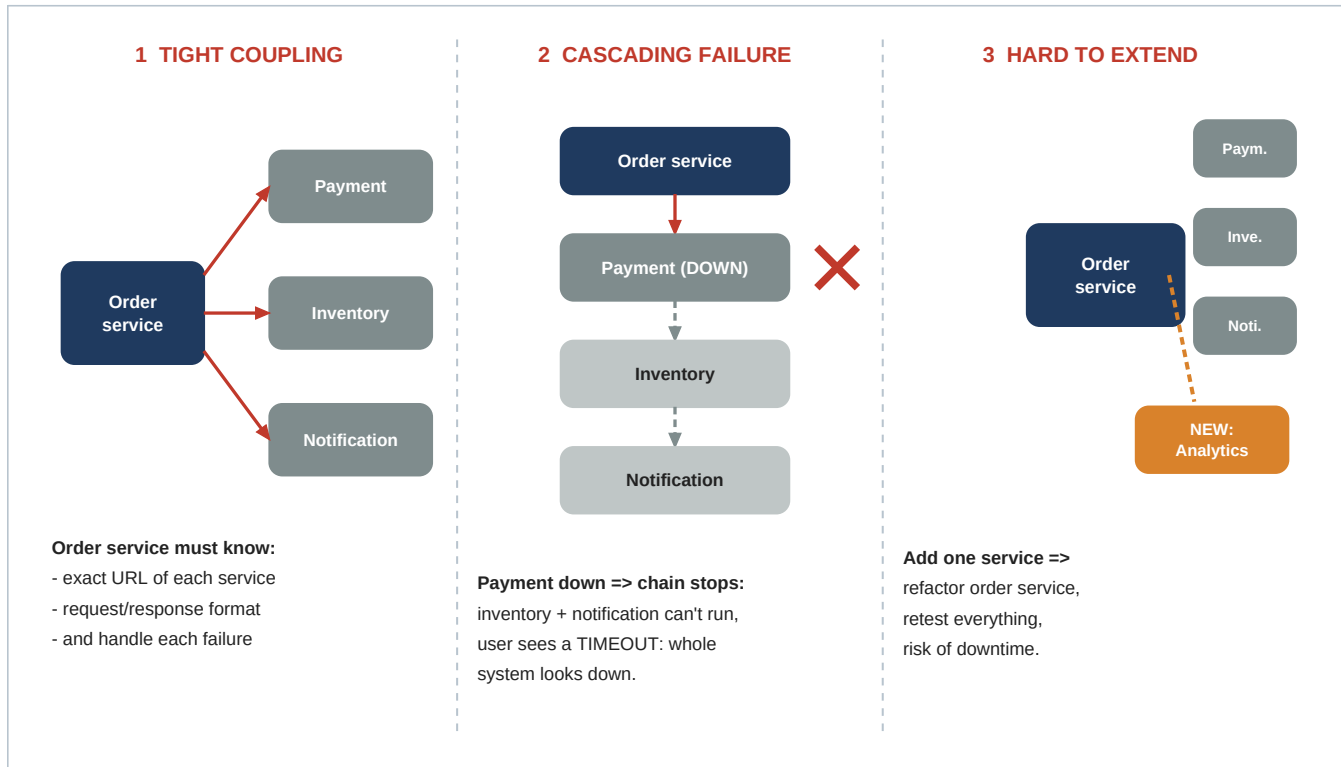


Figure 2: Tight coupling, cascading failure and hard-to-extend design

Problem 1: Tight coupling

The whole architecture is **tightly coupled**. The order service must know the **exact URL** of every other service, the **request and response format** of each, and it must **handle their failures**.

EXAMPLE

If the payment team changes its API from `/pay` to `/v2/charge` or renames a field, the order service breaks until its code is updated too.

Problem 2: Cascading failure

If the **payment service is down**, the order service's request is denied, the user gets a **timeout**, and it feels like the **entire system is down**, even though only payment is broken. Inventory and notification cannot even be called because the chain cannot proceed past payment.

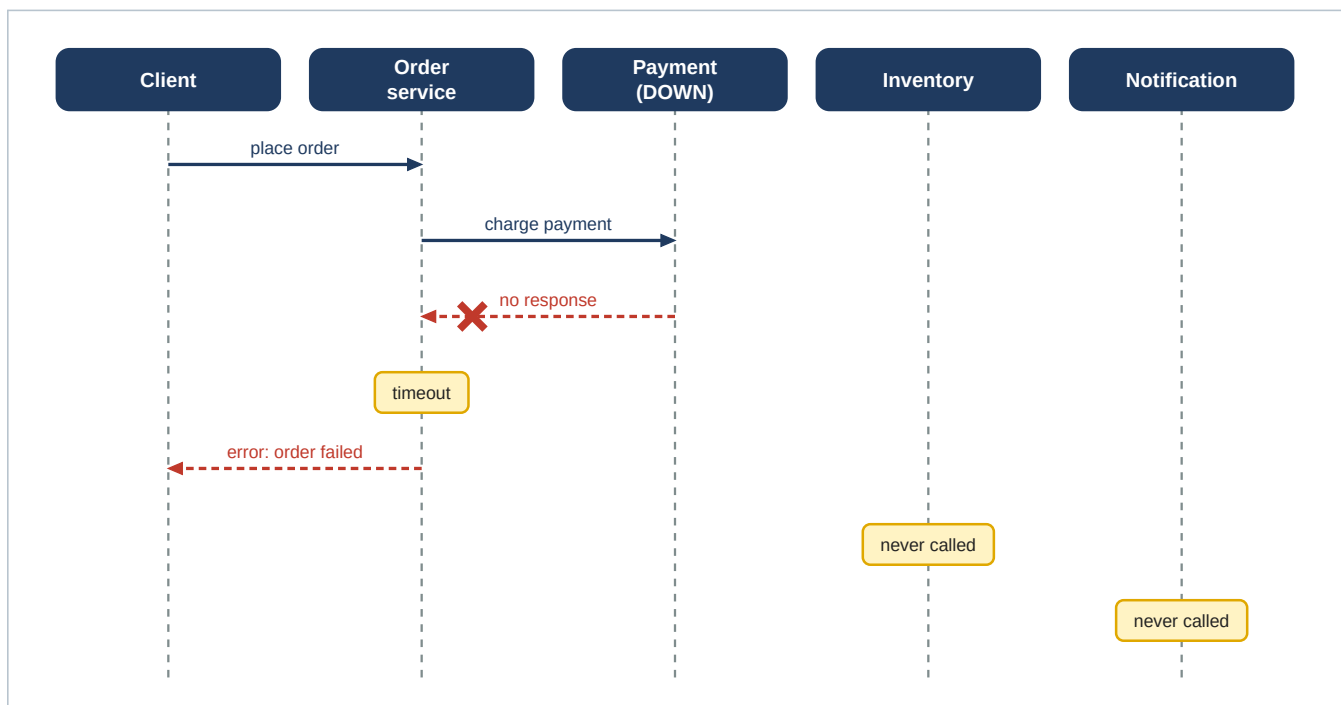


Figure 3: One dead service breaks the whole order flow

EXAMPLE

On a sale day the payment gateway has a 10-minute outage. Every order attempt fails at step 1, so customers also see 'could not place order', although inventory and email services are perfectly healthy.

Problem 3: Hard to extend

To add a new service, say **analytics**, the order service must learn how to communicate with it and handle its request and response format. That means **refactoring the whole order service**: take it down, change the code, test everything, bring it up. If testing is weak, the order service itself can fail.

EXAMPLE

Marketing wants every order sent to a recommendation engine and a fraud checker. Each request means another change and redeploy of the critical order service, with the risk of breaking checkout.

KEY TAKEAWAY

Tight coupling, cascading failure and poor extensibility are big challenges especially in distributed environments. Event-driven architecture was created to fix them.

2. What is Event-Driven Architecture?

Event-driven architecture is a software design principle in which each system component communicates by **producing an event** or **responding to an event**. Two services never call each other directly; they interact through events.

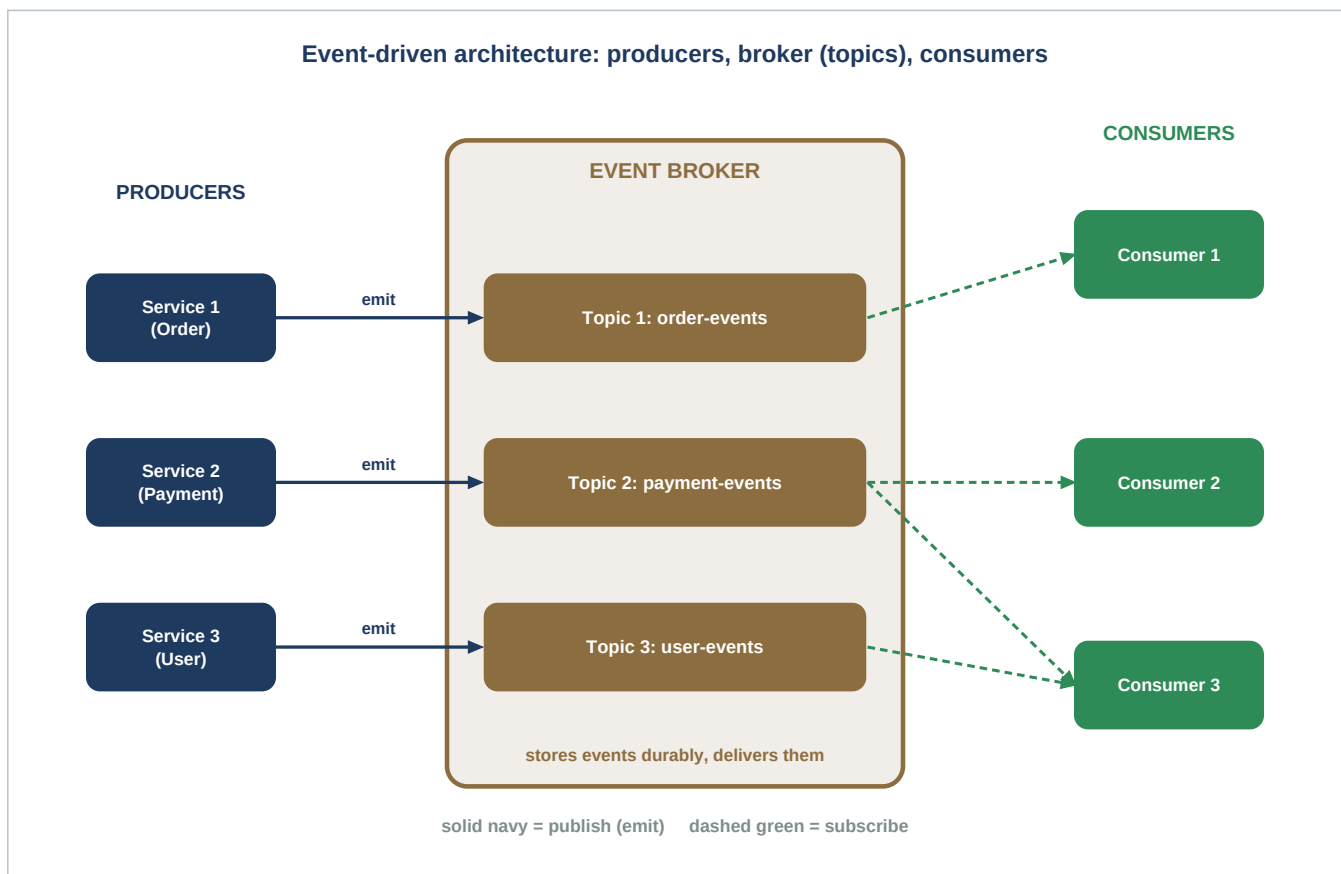


Figure 4: Producers publish events to topics in a broker; consumers subscribe

EXAMPLE

Instead of 'order service calls payment service', it becomes: order service announces 'an order was created'. Whoever cares (payment, inventory, analytics) reacts on their own.

2.1 What is an event?

An **event** is a **durable, immutable record of something that has already happened in the past**.

What is an event? A durable, immutable record of something that already happened

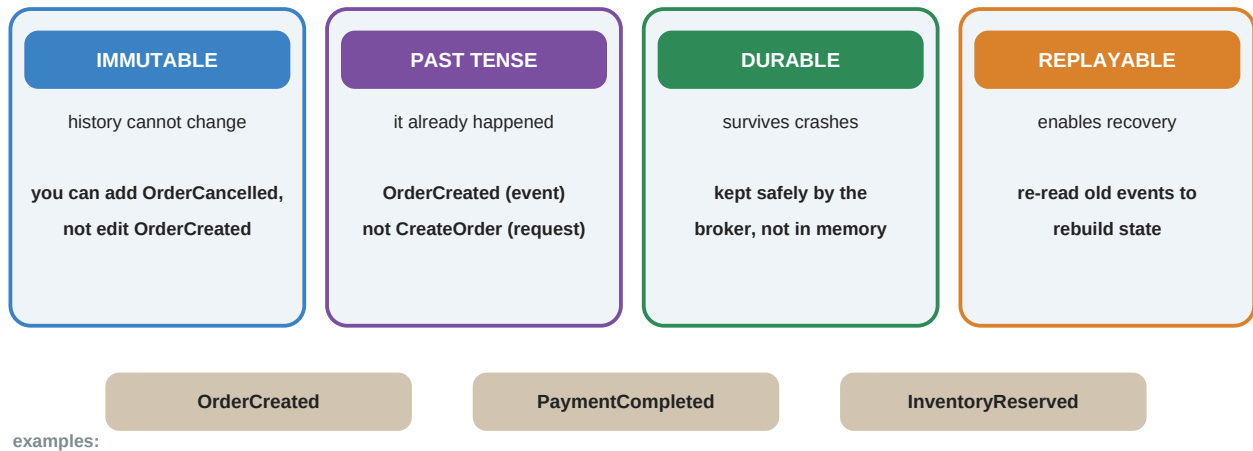


Figure 5: The four properties of an event

- **Immutable:** history cannot change. Example: if an order is cancelled you publish a new `OrderCancelled` event; you never edit the old `OrderCreated` event.
- **Past tense:** it describes something that already happened: `OrderCreated`, not `CreateOrder`.
- **Durable:** it can survive crashes (stored safely by the broker).
- **Replayable:** it enables recovery. You can replay events (read them again) but never change them. Example: a new consumer that crashed for an hour reads the events it missed.

Anatomy of a typical event



Figure 6: A typical event: a type in past tense plus the facts

EXAMPLE

Typical events: `OrderCreated`, `PaymentCompleted`, `InventoryReserved`.

3. The Four Components

An event-driven system has four major components: **event producer**, **event broker**, **event consumer** and the **topic** that connects them.

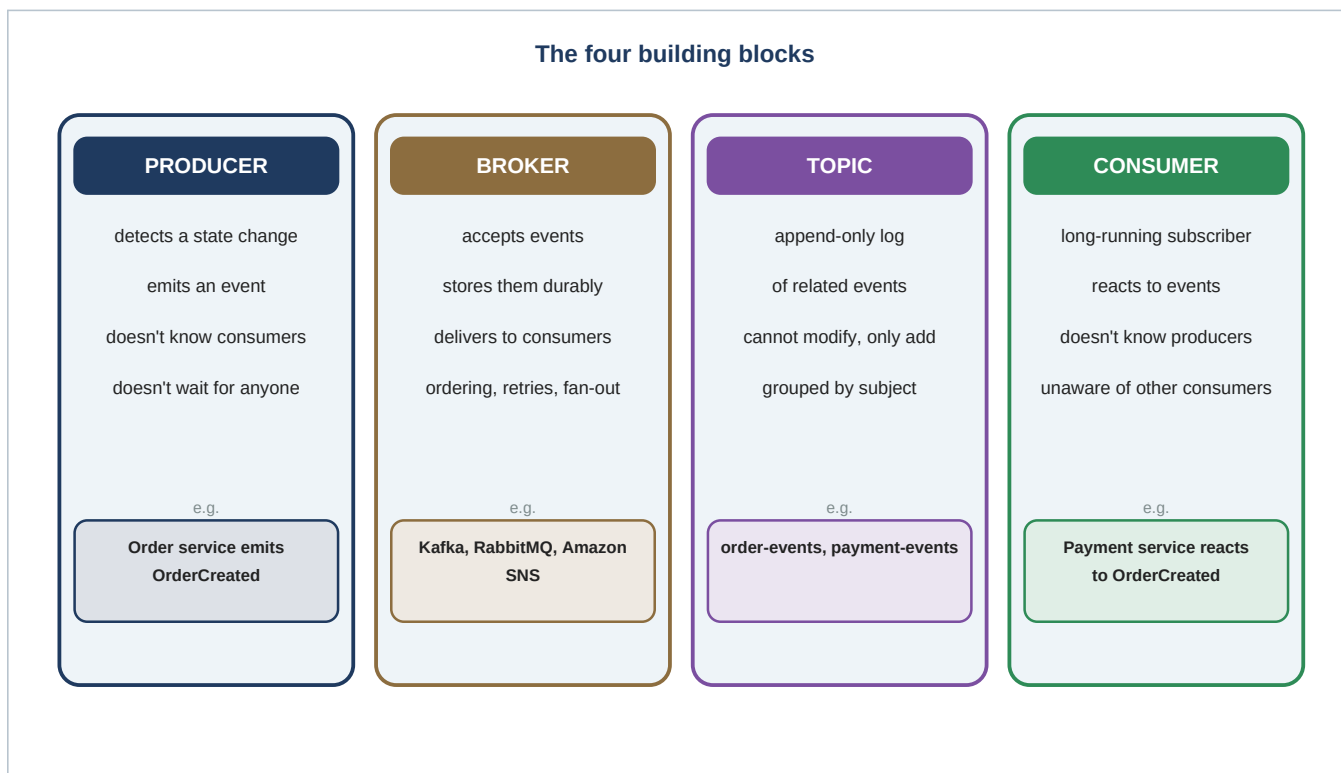


Figure 7: Producer, broker, topic and consumer at a glance

3.1 Event producer

An **event producer** is any service that **detects a state change** and **emits an event describing it**.

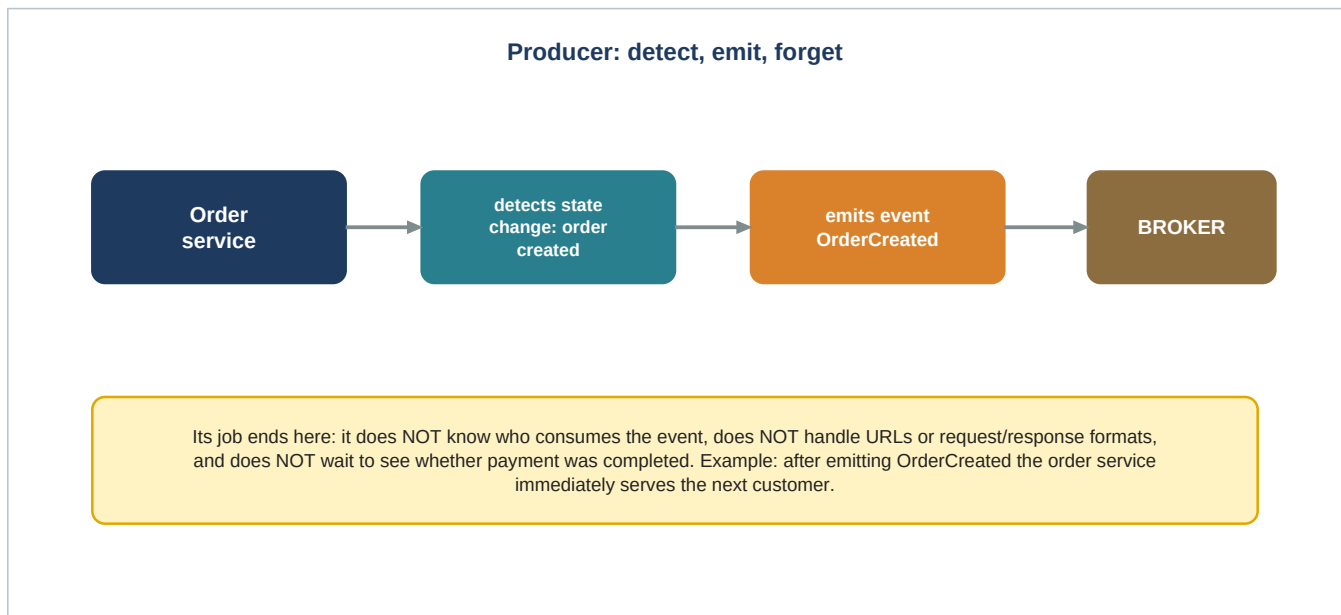


Figure 8: A producer detects a change, emits an event and moves on

EXAMPLE

The order service creates an `OrderCreated` event when you place an order. A payment service emits a `PaymentCompleted` event whenever it accepts a payment.

Two important facts (popular interview points):

- **The producer does not know who consumes the event.** It creates the event and sends it to the broker; it does not care about the consumers, their URLs or their request/response formats.
- **The producer does not wait.** It does not wait to see whether the payment was completed or verified. It detects a change, emits an event and is done.

3.2 Event broker: the heart of EDA

The **event broker** accepts events from producers, **stores them durably**, **delivers** them to consumers and manages **ordering**, **retries** and **fan-out**. Examples: **Kafka**, **RabbitMQ**, **Amazon SNS** (some are push-based, some pull-based).

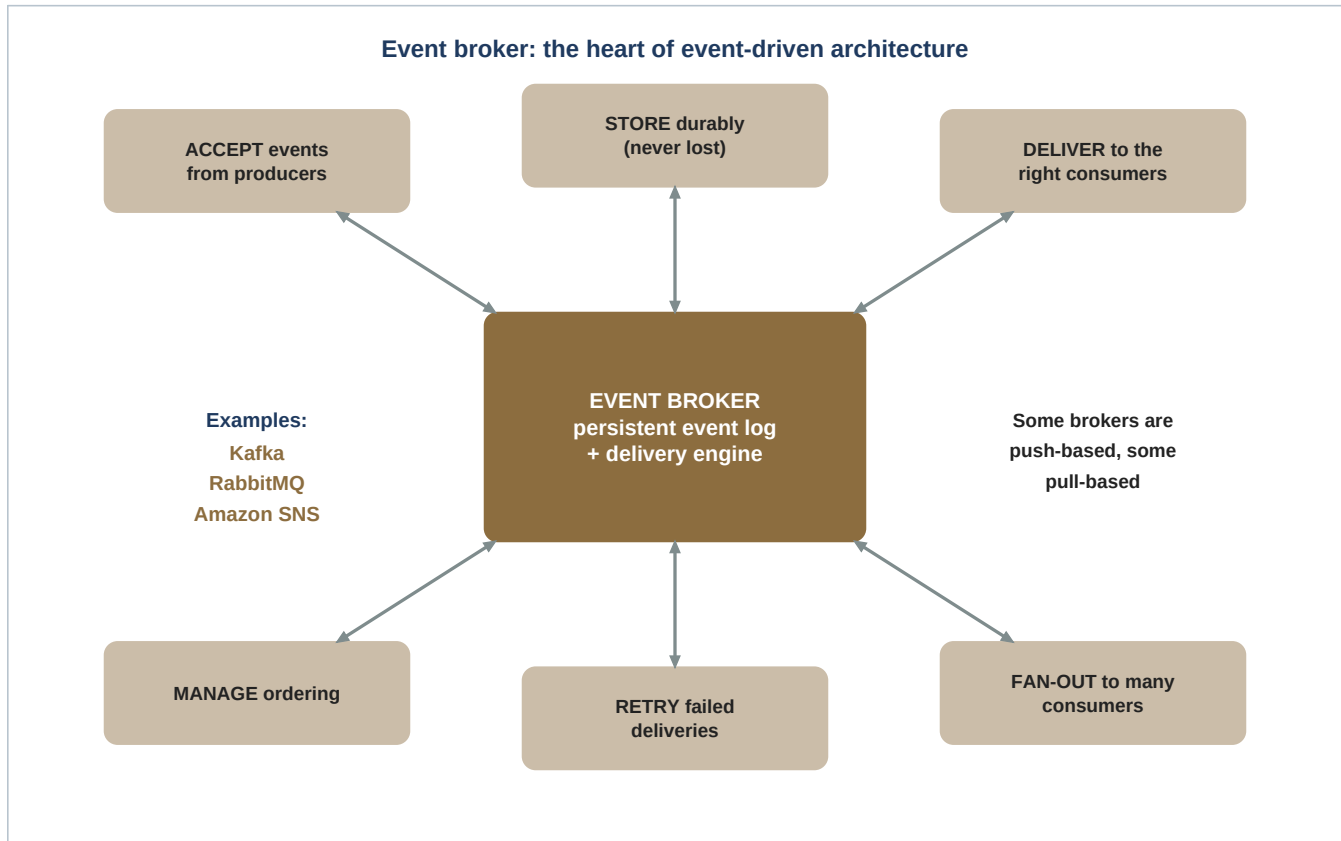


Figure 9: Everything between producer and consumer is the broker's job

- **Accept:** the producer emits and forgets; the broker takes over.
- **Store durably:** an event must not be lost at any cost.
- **Deliver:** it passes each event to the right consumers.
- **Ordering:** events are delivered in a sensible order.
- **Retries:** if delivery fails it tries again.
- **Fan-out:** one event can go to many consumers.

EXAMPLE

Think of the broker as a post office: you drop a letter and leave; the post office stores it safely, sorts it, delivers it to everyone on the mailing list and re-attempts if nobody was home. In short, a broker is a **persistent event log plus a delivery engine**.

3.3 Event consumer

An **event consumer** is a **long-running subscriber** to one or more topics that **reacts to events**. It waits for events and, when one arrives, does the required processing.

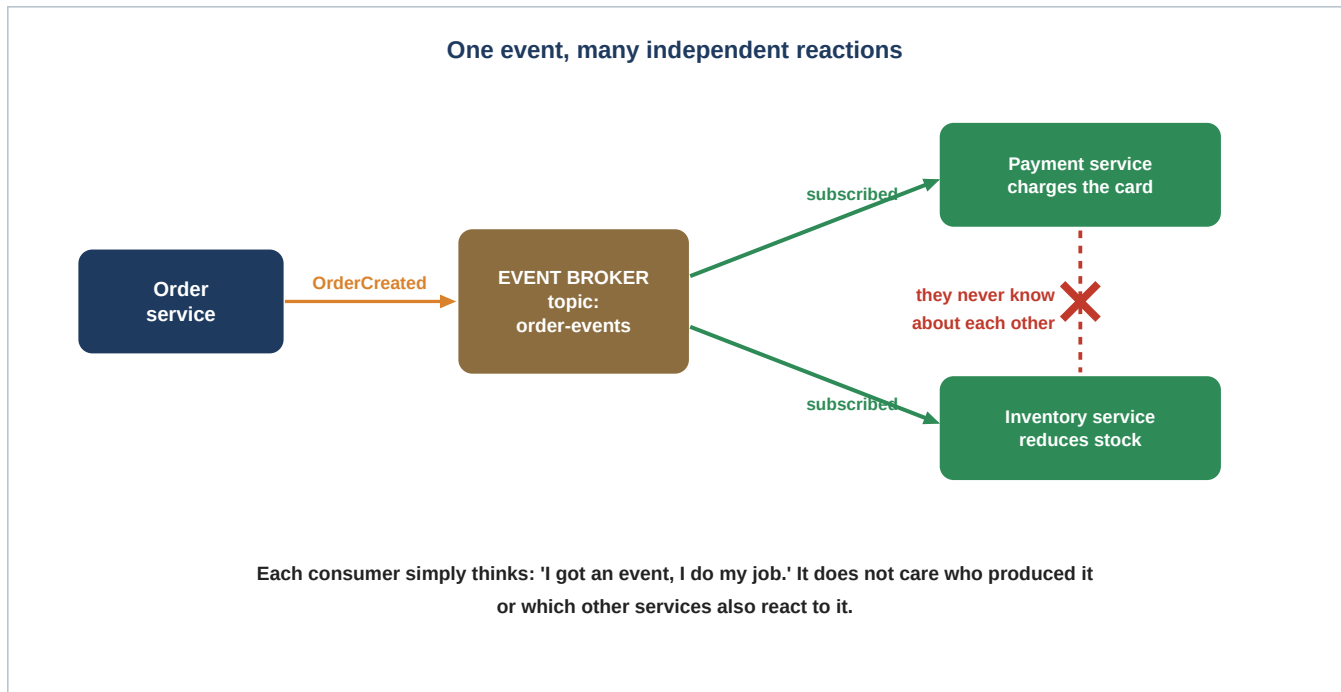


Figure 10: Payment and inventory both react to *OrderCreated*, independently

EXAMPLE

Payment service consumes *OrderCreated* and charges the user. **Inventory service** consumes the same event and reduces stock for the ordered item. Both react at their own pace.

- A consumer **does not know who the producer is**.
- Consumers **do not know about each other**: payment does not know inventory exists and vice versa. They all depend only on the broker.
- Each just thinks 'I got an event, I do my job'.

3.4 Topic

A **topic** is a **log-like, append-only stream of related events**. You can only **add** events to it, never modify them. Examples: *order-events*, *payment-events*, *user-events*.

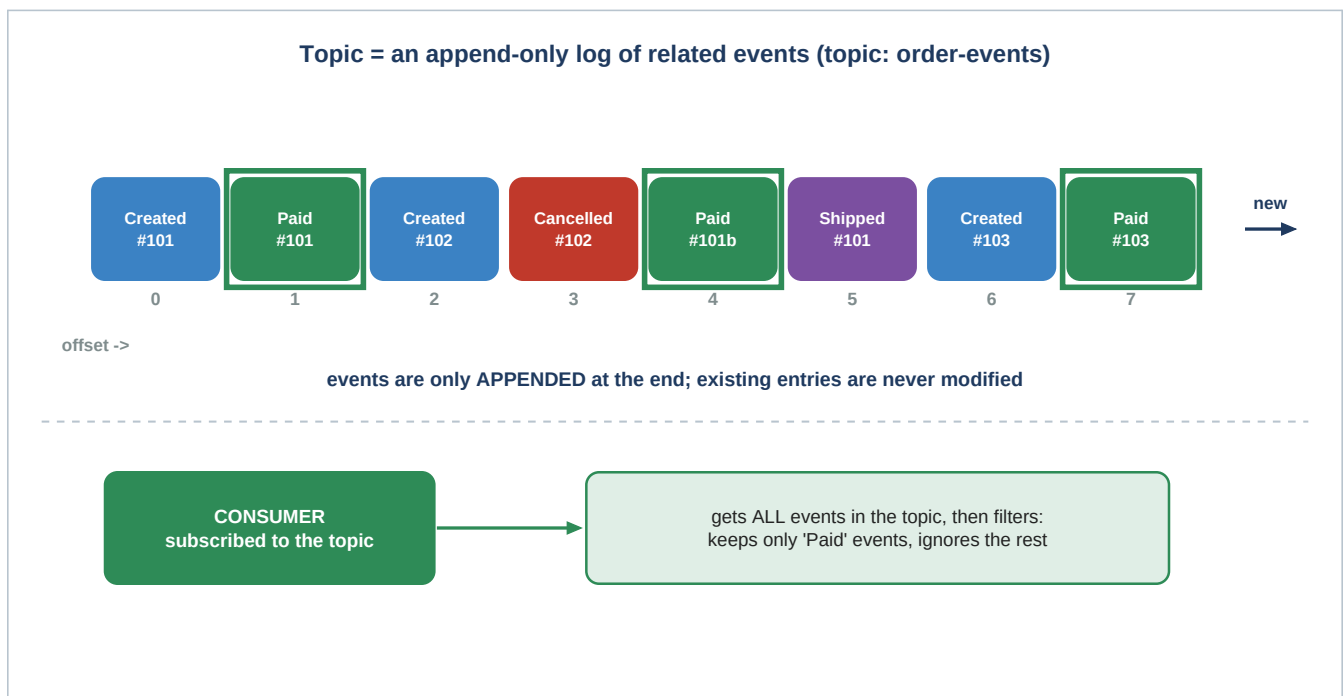


Figure 11: A topic is an append-only log; a consumer reads it and filters

EXAMPLE

The *order-events* topic contains OrderCreated, OrderPaid, OrderCancelled and OrderShipped events, all grouped because they concern orders. A consumer that only handles payments subscribes to the topic, receives all events, then **filters** to keep only the OrderPaid ones.

3.5 The checkout flow, redesigned

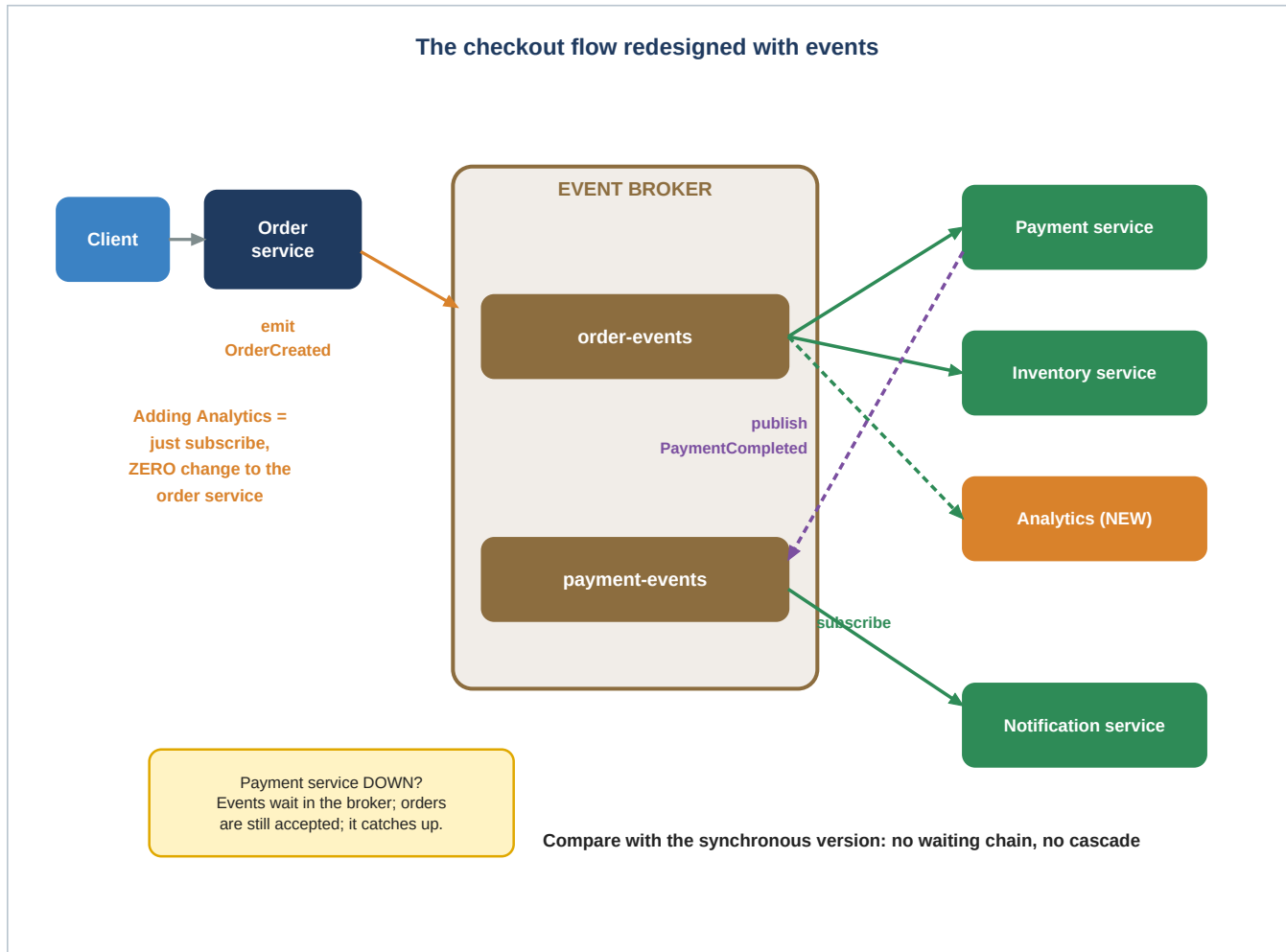


Figure 12: Order flow with events: decoupled, resilient and easy to extend

- **Loose coupling:** the order service just emits OrderCreated; it knows nothing about payment or inventory.
- **No cascading failure:** if payment is down, events wait in the broker, orders are still accepted, and payment catches up on restart.
- **Easy to extend:** analytics simply subscribes to order-events; the order service is untouched.
- **Chaining:** payment publishes PaymentCompleted; notification subscribes to payment-events and sends the email.

Aspect	Synchronous (chain of calls)	Event-driven
Coupling	Tight (URLs, formats, failures)	Loose (only the broker and event shapes)
If payment is down	Whole order fails, user sees timeout	Events wait; order accepted; catch up later
Adding analytics	Refactor and redeploy order service	Subscribe a new consumer
Waiting	Caller waits for every step	Producer emits and moves on
Scalability	Limited by the slowest service	Consumers scale independently

4. Event Processing Patterns

An **event processing pattern** describes a common way to **handle, transform and react to events**. The video covers three.

4.1 Event notification

The event carries **minimal information**, just enough to say that something happened. Example: `{ type: OrderCreated, orderId: 123 }`. The event is **small and sent quickly**, but the consumer needs **extra calls** (for example to the database or the source service) to fetch more information.

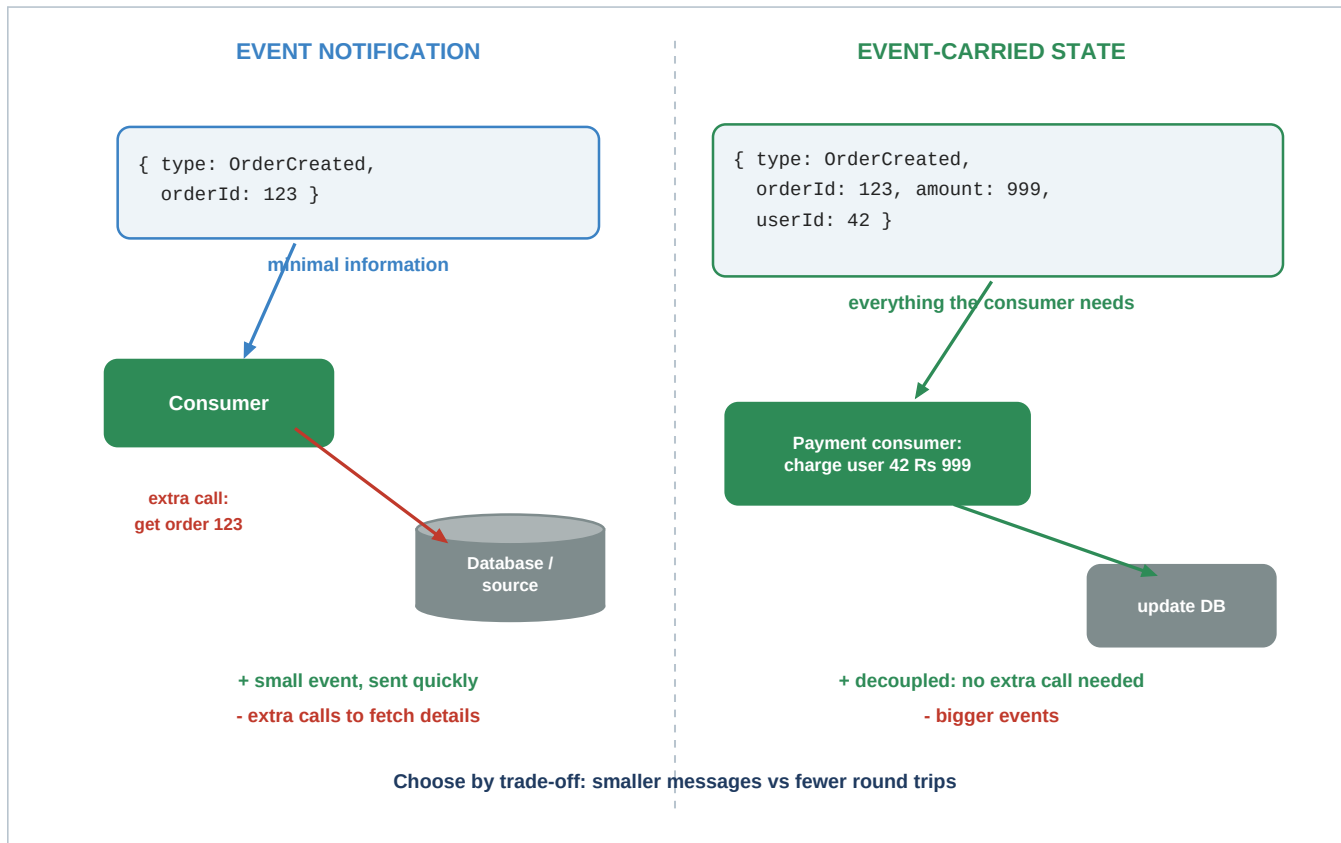


Figure 13: Event notification vs event-carried state

EXAMPLE

The notification consumer gets 'OrderCreated #123'. To write the email it asks the order system for the customer's name and items: one more round trip per event.

4.2 Event-carried state

The event carries **everything the consumer needs**. Example: `{ type: OrderCreated, orderId: 123, amount: 999, userId: 42 }`. No extra call is needed, so consumers are **decoupled** and need not wait for other events; the trade-off is a **larger event**.

EXAMPLE

The payment consumer receives the amount and user ID inside the event, so it can charge user 42 Rs 999 and update its database immediately, without asking anyone for details.

Aspect	Event notification	Event-carried state
Event content	Minimal (type + id)	Everything needed
Event size	Small, fast	Larger
Extra calls	Yes (fetch details)	No
Coupling to source	Higher (callbacks)	Lower (self-contained)

4.3 Event sourcing

In event sourcing you **store the events, not the state**; the state is **derived** by replaying the events.

A traditional database stores only the **current state**: order status = SHIPPED. If someone asks 'how did it reach SHIPPED?' or 'what was the status yesterday?', you cannot answer because each update overwrote the previous one.

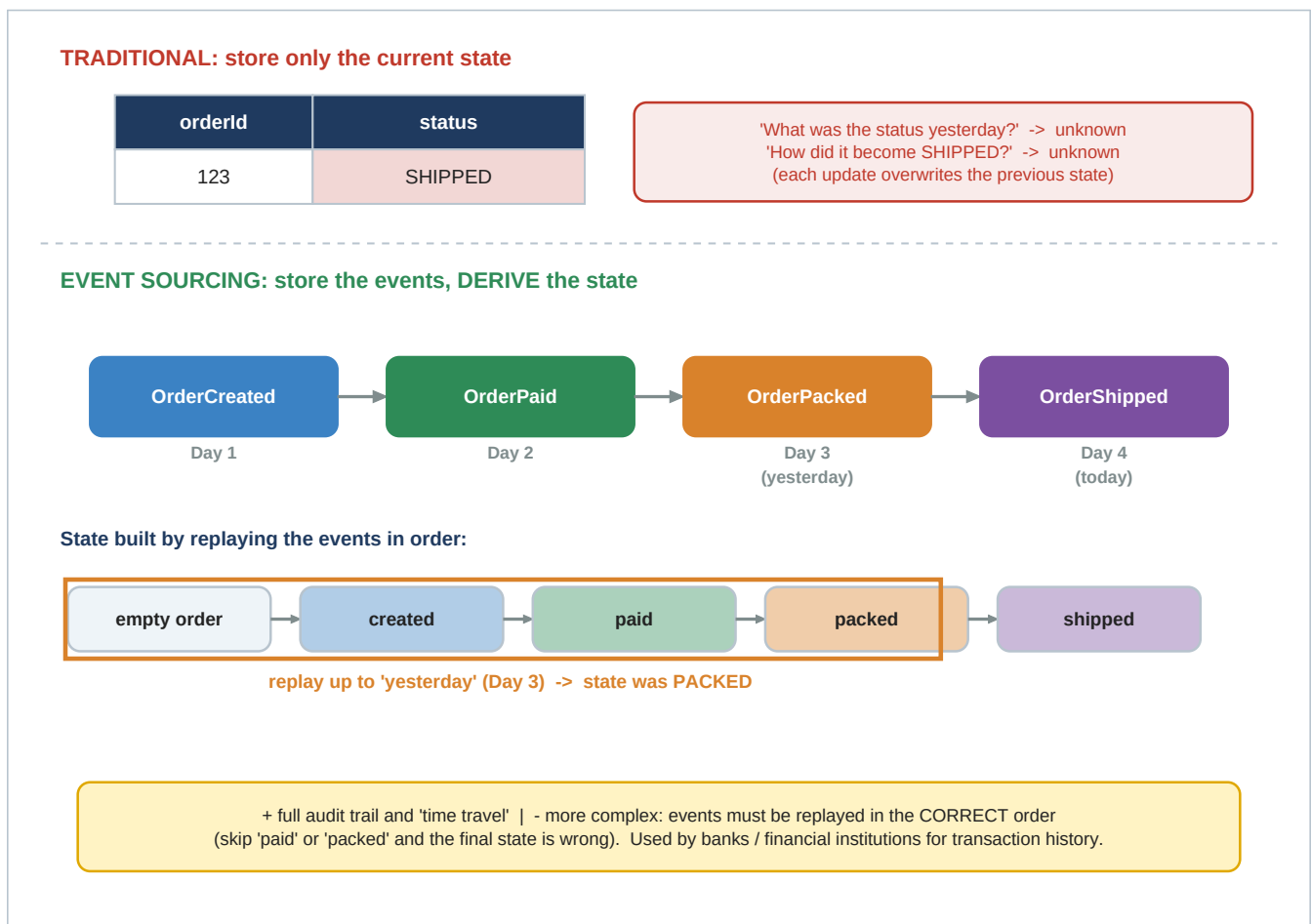


Figure 14: State derived from a stored event history

EXAMPLE

Events: OrderCreated (Day 1), OrderPaid (Day 2), OrderPacked (Day 3), OrderShipped (Day 4). Replaying all gives today's state: shipped. Asked 'what was the state yesterday?', replay up to Day 3 and the answer is: packed.

- **Benefits:** a **full audit trail** and **time travel**: you can answer any question about the past.

- **Cost:** the design becomes **complex**. Events must be replayed in the **correct order**; skipping 'paid' or 'packed' would lead to an incorrect final state.
- **Where it is used:** **banks and financial institutions** that must track an entire transaction history: what was credited, when it was created, how much was transferred, where, and the final balance.

5. Failure Handling and Delivery Semantics

5.1 Where failures can happen

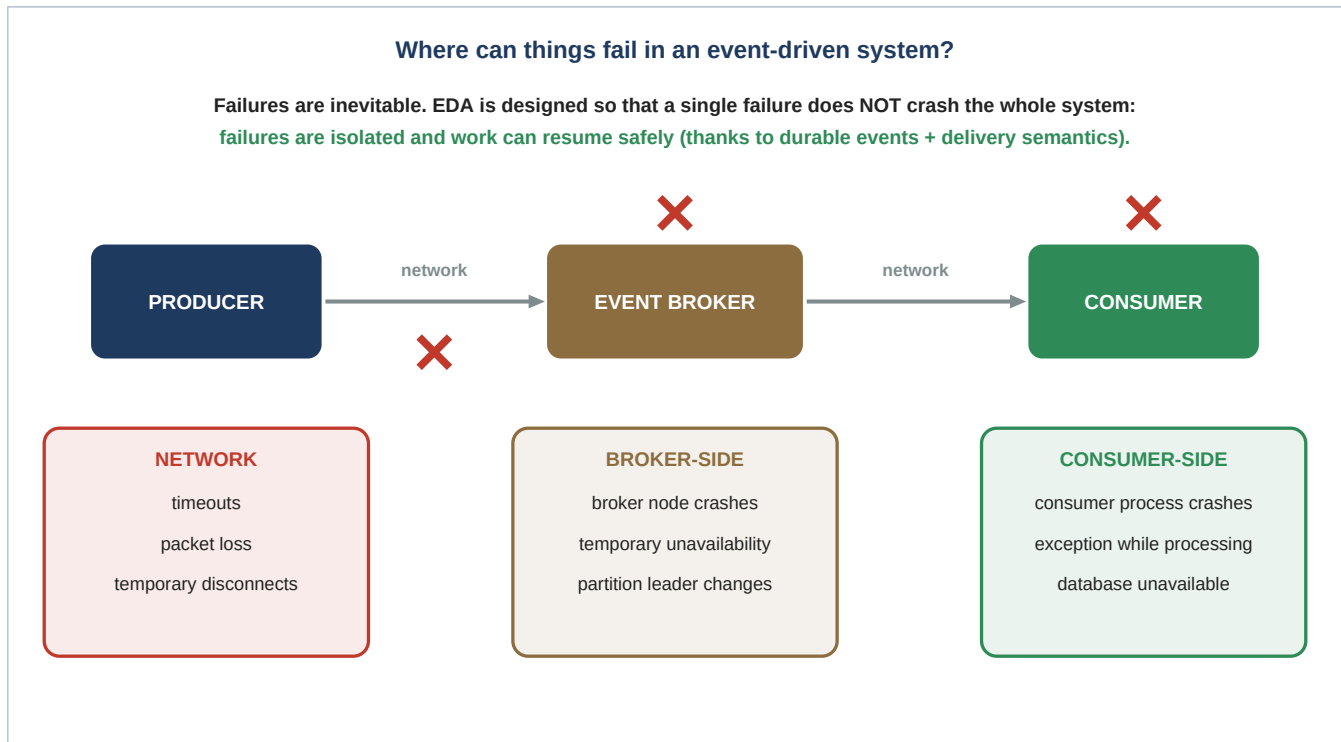


Figure 15: Consumer-side, broker-side and network failures

- **Consumer-side:** the consumer process crashes, an exception occurs while processing, or the database is unavailable.
- **Broker-side:** a broker node crashes, is temporarily unavailable, or a partition leader changes.
- **Network:** timeouts, packet loss, temporary disconnects.

Failures are **inevitable**; you cannot avoid them. EDA is designed so that a **single failure does not crash the whole system**: failures are **isolated** and work can **resume safely**. The tool for this is **delivery semantics**, which define how the system behaves when something fails.

EXAMPLE

The email consumer crashes for 5 minutes. Orders keep flowing, payments and stock updates continue, and when the email consumer restarts it reads the events it missed from the broker and sends the emails. Nothing else noticed.

5.2 At-most-once delivery

Each event is delivered **zero or one time**. It **may be lost but is never duplicated**. The broker sends the event and **does not wait for an acknowledgement**; it assumes it was processed. There are **no retries**.

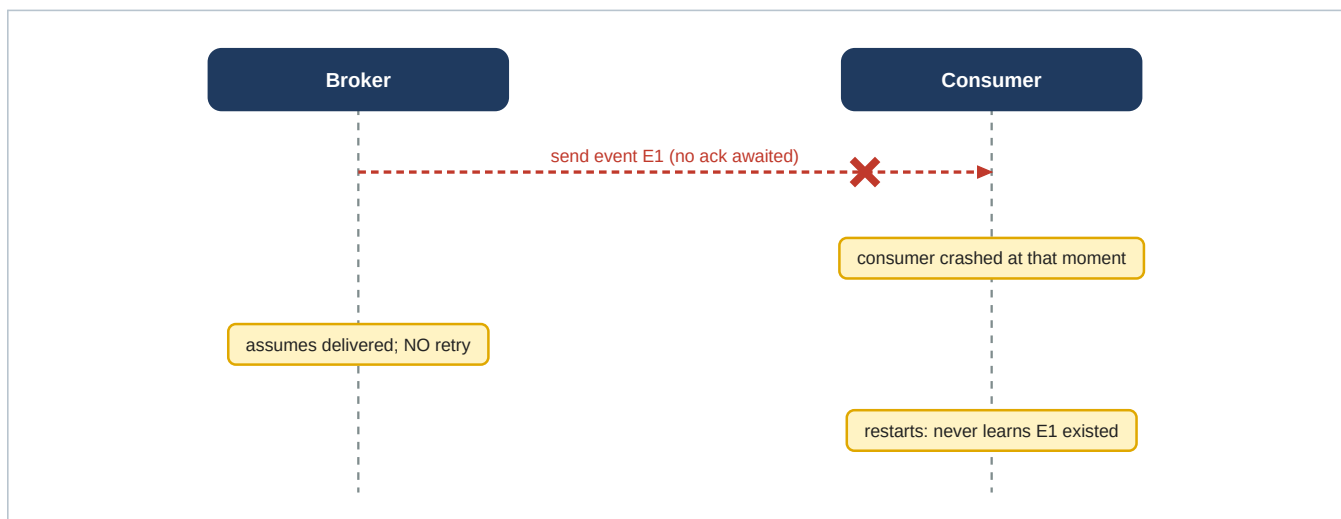


Figure 16: At-most-once: if the consumer crashes, the event is gone

EXAMPLE

Logs and metrics: losing one or two log lines does not matter, so the speed and simplicity of at-most-once is a good trade. It would be wrong for payments.

5.3 At-least-once delivery

Each event is delivered **one or more times**. Events are **never lost, but duplicates are possible**. The broker keeps re-sending until the consumer **acknowledges**. It is used in real-time systems and is the **default in EDA**.

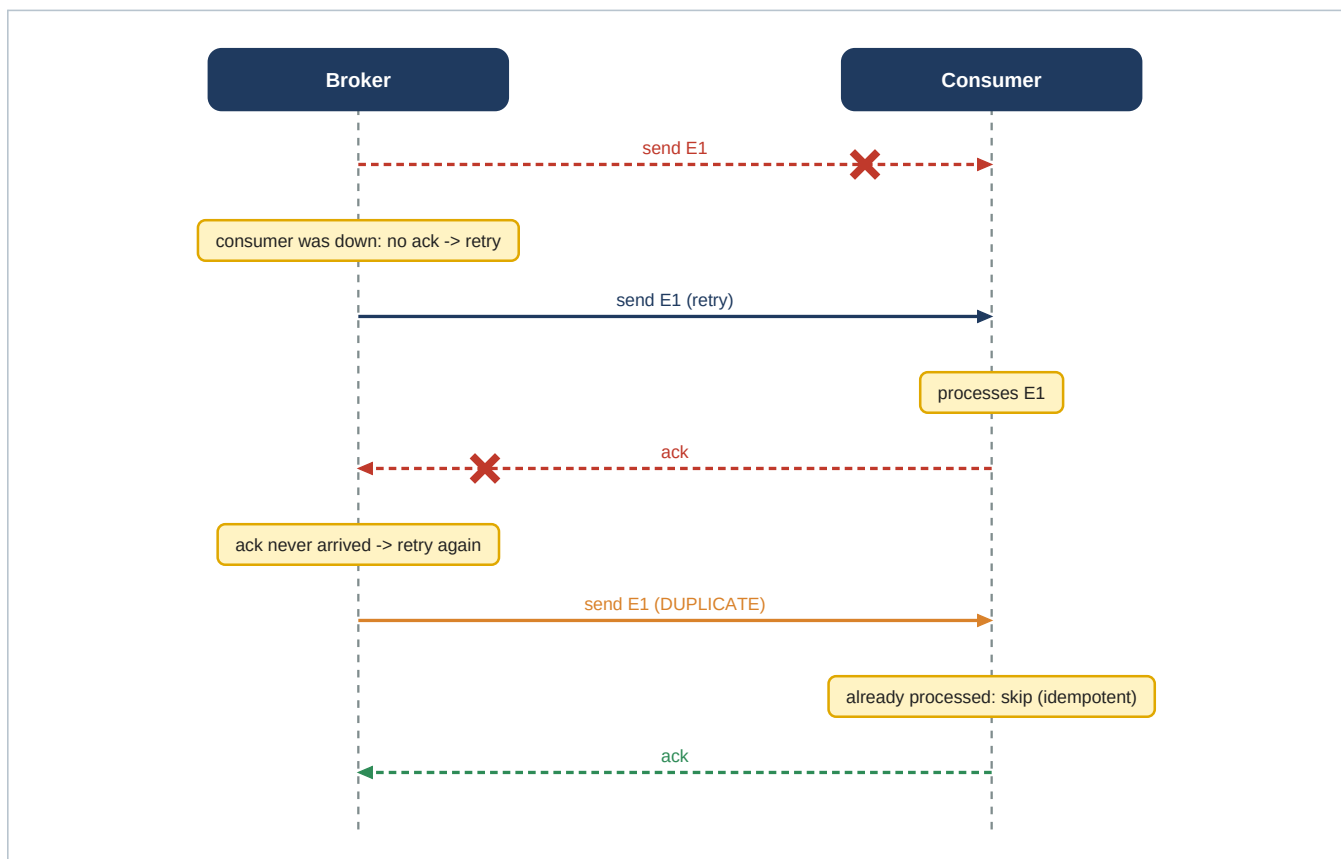


Figure 17: At-least-once: retries guarantee delivery but can create duplicates

EXAMPLE

The broker sends 'PaymentCompleted #77'. The consumer was down, so no ack: retry. This time the consumer processes it, but its ack is lost on the network, so the broker retries again and the consumer receives a duplicate.

Making at-least-once safe: idempotent consumers

To make at-least-once safe, the **consumer must be idempotent**: it should handle duplicates so that receiving the same event twice leaves the final state exactly as if it had arrived once.

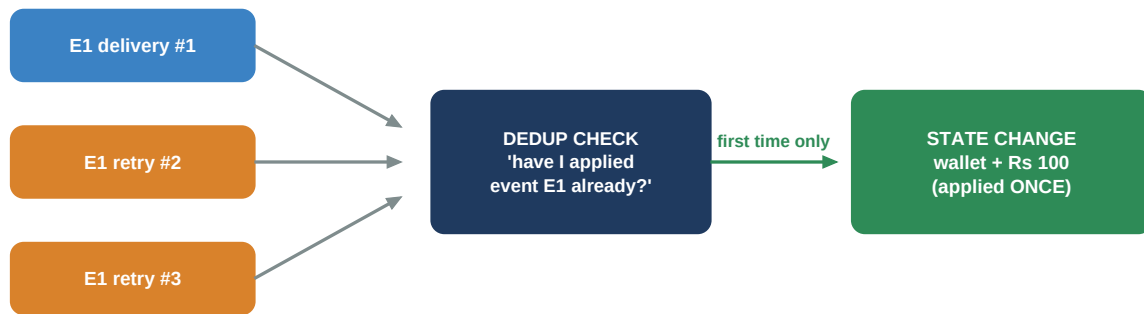
EXAMPLE

The consumer keeps a table of processed event IDs. When duplicate #77 arrives it sees '#77 already applied' and skips it, so the customer is charged once, not twice.

5.4 Exactly-once delivery

Each event affects the **system state exactly once, even in the presence of failures**: it is not lost (unlike at-most-once) and not duplicated in effect (unlike at-least-once). Typical uses: **financial settlements and wallet balances** without duplication.

Exactly-once effect: duplicates may arrive, but the state changes once



Balance after 3 deliveries: Rs 100 (not Rs 300)

Typical for financial settlements and wallet balances

(in practice built from at-least-once delivery + idempotent / transactional processing)

Figure 18: Three deliveries of the same event, one state change

EXAMPLE

Wallet at Rs 0. Event 'credit Rs 100' is delivered three times because of retries. After exactly-once processing the balance is Rs 100, not Rs 300.

GOOD TO KNOW

Exactly-once is the strongest and most expensive guarantee. In practice it is usually achieved by combining at-least-once delivery with idempotent or transactional processing (deduplication), as in the figure; use it only where correctness demands it.

5.5 Comparison

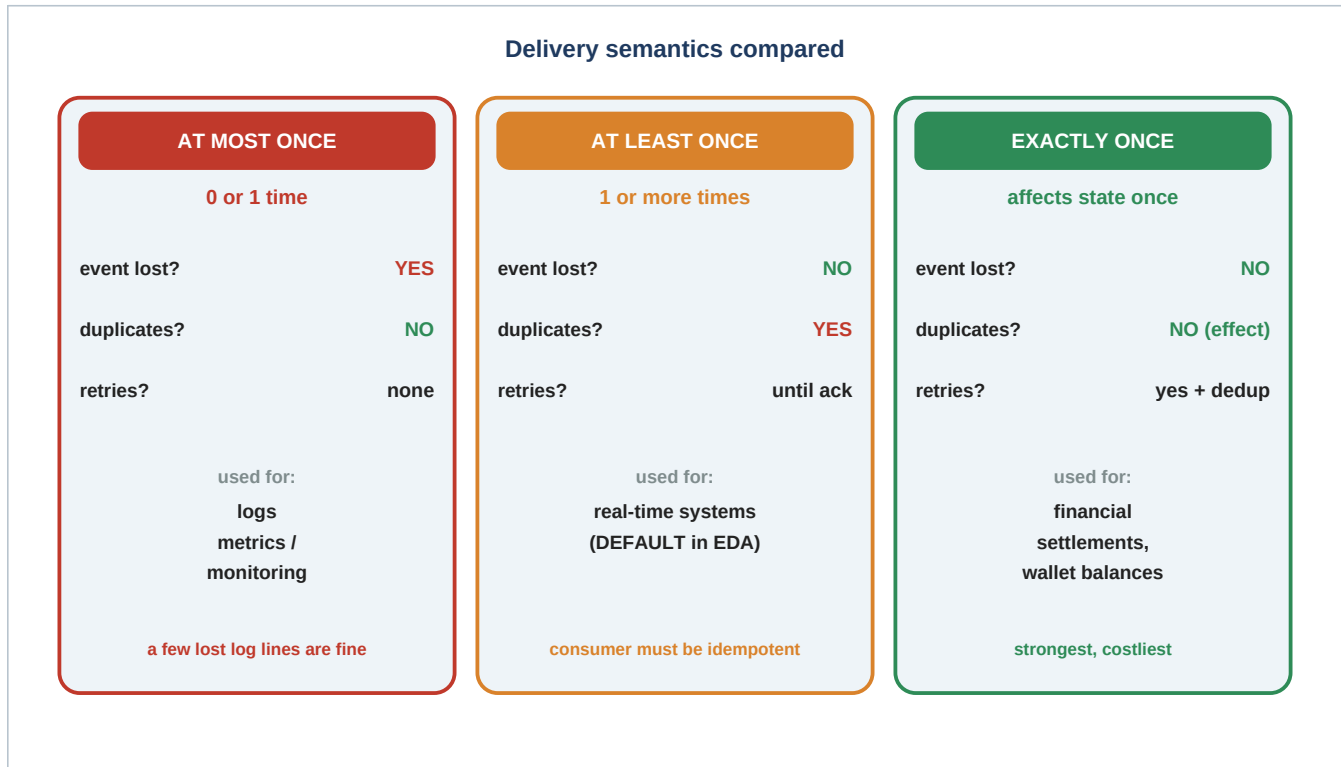


Figure 19: At-most-once vs at-least-once vs exactly-once

Semantics	Lost events?	Duplicates?	Retries	Typical use
At most once	Possible	Never	None	Logs, metrics, monitoring
At least once	Never	Possible	Until acknowledged	Real-time systems; default in EDA (needs idempotent consumers)
Exactly once	Never	No duplicate effect	Yes + dedup	Financial settlements, wallet balances

6. EDA in Real Systems

EDA is used where a **single action must trigger multiple independent reactions reliably, at scale, and without tight coupling**. If you do not need these properties, you do not need EDA.

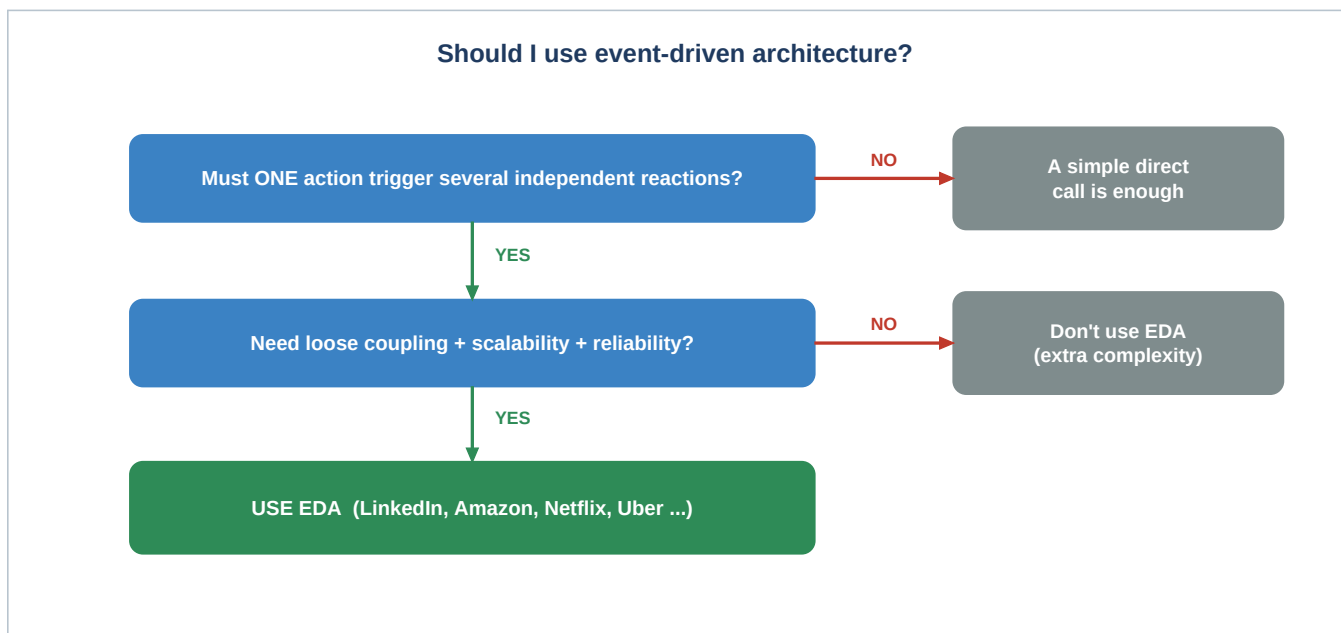


Figure 20: A quick decision flow for choosing EDA

6.1 LinkedIn

LinkedIn uses event-driven architecture for its **feed, notification and recommendation services**. A post service emits an event to the broker; the news feed, recommendation and notification services are all subscribed.

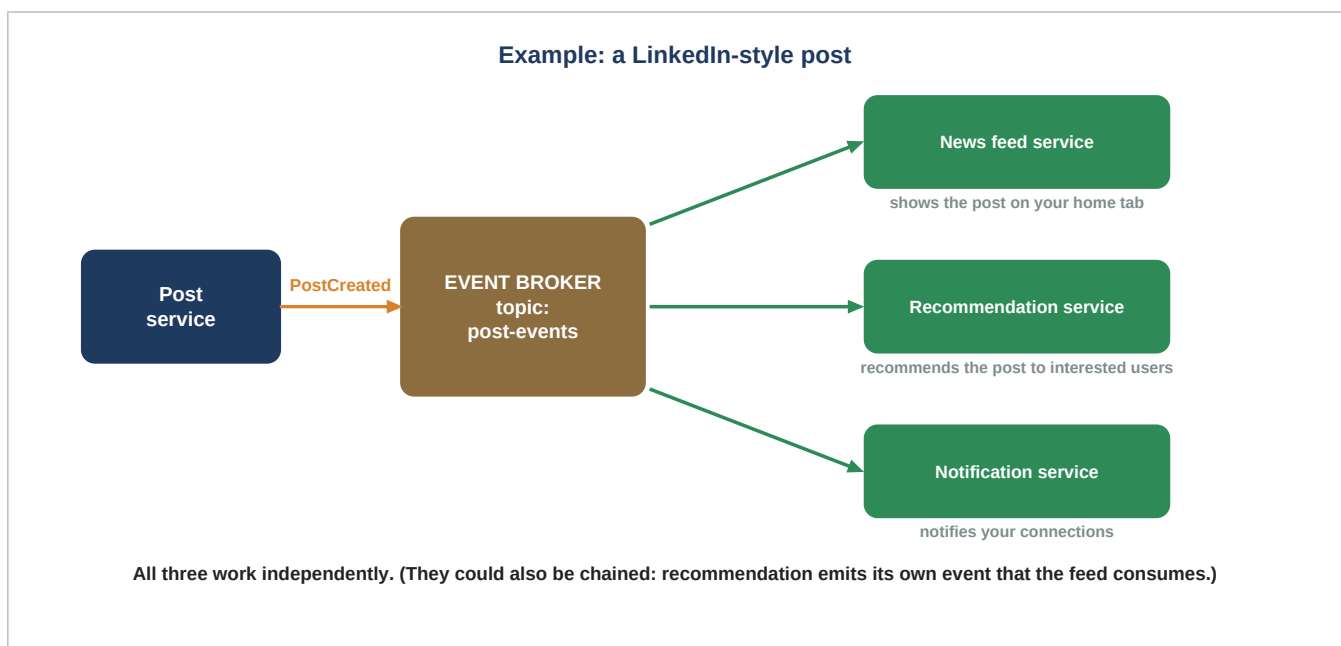


Figure 21: One post event, three independent reactions

EXAMPLE

You publish a post. Notification service tells your connections, the recommendation service starts suggesting it to people who might care, and the news feed service shows it on home tabs when they refresh. The recommendation and feed services could also be chained: recommendation processes the event, emits a 'recommendations changed' event, and the feed consumes that.

6.2 Others

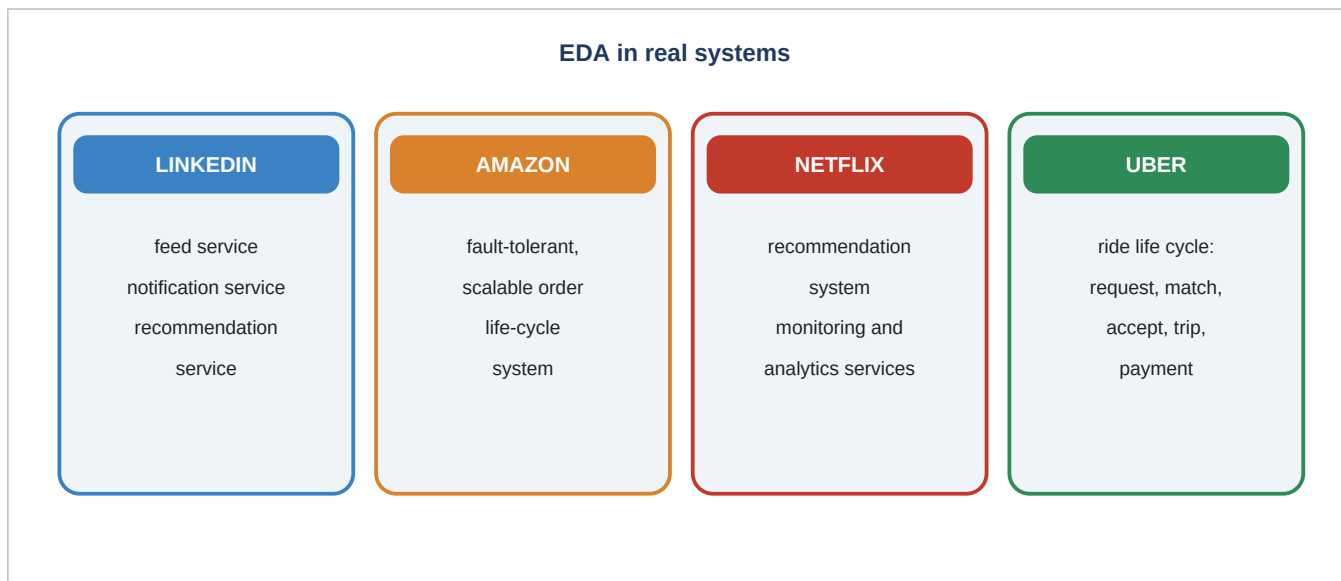


Figure 22: EDA at LinkedIn, Amazon, Netflix and Uber

- **Amazon:** builds a fault-tolerant, scalable **order life-cycle system** with events.
- **Netflix:** powers its **recommendation system** and its **monitoring and analytics services**.
- **Uber:** uses EDA for the **ride life cycle**.

KEY TAKEAWAY

Use EDA when you need loose coupling, scalability and independent reactions. Otherwise a simple synchronous call is usually enough.

GOOD TO KNOW (NOT IN THE VIDEO)

EDA is not free: debugging a flow spread across many consumers is harder, results are eventually consistent (the email may arrive a moment after the payment), and you must design for duplicates and ordering. That is the price of decoupling.

7. Cheat Sheet and Interview Q&A

7.1 One line per topic

Topic	Remember this
Synchronous problems	Tight coupling, cascading failure, hard to extend.
EDA	Components communicate by producing or responding to events.
Event	Durable, immutable, past-tense, replayable record of something that happened.
Producer	Detects a state change, emits an event; doesn't know consumers; doesn't wait.
Broker	Heart of EDA: accepts, stores durably, delivers, ordering, retries, fan-out (Kafka, RabbitMQ, SNS).
Consumer	Long-running subscriber that reacts to events; doesn't know producers or other consumers.
Topic	Append-only log of related events, e.g. order-events; consumers filter what they need.
Event notification	Minimal event; small and fast; needs extra calls.
Event-carried state	Event carries everything; decoupled; bigger.
Event sourcing	Store events, derive state; audit trail + time travel; complex; banks.
At most once	May lose, never duplicates, no retries (logs, metrics).
At least once	Never loses, may duplicate; default; consumer must be idempotent.
Exactly once	State changes exactly once; financial settlements, wallets.
Real usage	LinkedIn, Amazon, Netflix, Uber.

7.2 Interview Q&A

Q: What problems does synchronous architecture have?

A: Tight coupling, cascading failure when one service is down, and difficulty adding new services.

Q: What is event-driven architecture?

A: A design where components communicate by producing events or responding to events through a broker.

Q: What is an event?

A: A durable, immutable record of something that happened in the past, e.g. OrderCreated.

Q: Why must events be immutable?

A: So history can't be rewritten; corrections are new events. This keeps replay and audit reliable.

Q: Does a producer know its consumers?

A: No. It emits to the broker and doesn't wait or care who consumes.

Q: Why is the broker the heart of EDA?

A: It stores events durably and handles delivery, ordering, retries and fan-out between producers and consumers.

Q: Name some brokers.

A: Kafka, RabbitMQ, Amazon SNS (push or pull based).

Q: Do consumers know each other?

A: No. They only depend on the broker and the events they subscribe to.

Q: What is a topic?

A: An append-only log of related events (e.g. order-events); consumers subscribe and filter.

Q: Event notification vs event-carried state?

A: Notification: minimal event, extra calls to fetch details. Carried state: event holds everything, decoupled but larger.

Q: What is event sourcing and where is it used?

A: Store events instead of state and derive state by replay; gives audit trail and time travel; used in banks.

Q: Downside of event sourcing?

A: Complexity: events must be replayed in correct order or the state is wrong.

Q: Where can failures occur in EDA?

A: Consumer side, broker side, and the network.

Q: Explain at-most-once.

A: 0 or 1 delivery, may be lost, no duplicates, no retries; fine for logs/metrics.

Q: Explain at-least-once and its requirement.

A: 1 or more deliveries, no loss, possible duplicates; consumers must be idempotent.

Q: Explain exactly-once.

A: Each event affects state exactly once even with failures; for financial settlements and wallet balances.

Q: When should you use EDA?

A: When one action triggers multiple independent reactions and you need loose coupling and scalability.

7.3 Notes on the source material

- The transcript is auto-generated, so some words are garbled: 'rabbit / aid MQ' = RabbitMQ, 'inventory reversed event' = InventoryReserved, 'at most one / at least one' = at-most-once / at-least-once, 'ID potent' = idempotent.
- The video says an event 'will be sent to the right consumers' by the broker and that brokers 'never crash'; in practice brokers are made durable and highly available through replication, but failures remain possible, which is why the video then discusses failure handling.
- In 'event notification' the extra calls are described as calls to the database; they can equally be calls to the producing service's API.
- The video says EDA is not needed when its benefits are not required; the 'GOOD TO KNOW' boxes add trade-offs (debugging, eventual consistency) that are common interview follow-ups but were not covered in the video.
- The Kafka/RabbitMQ/SNS internals are explicitly deferred to a separate video.